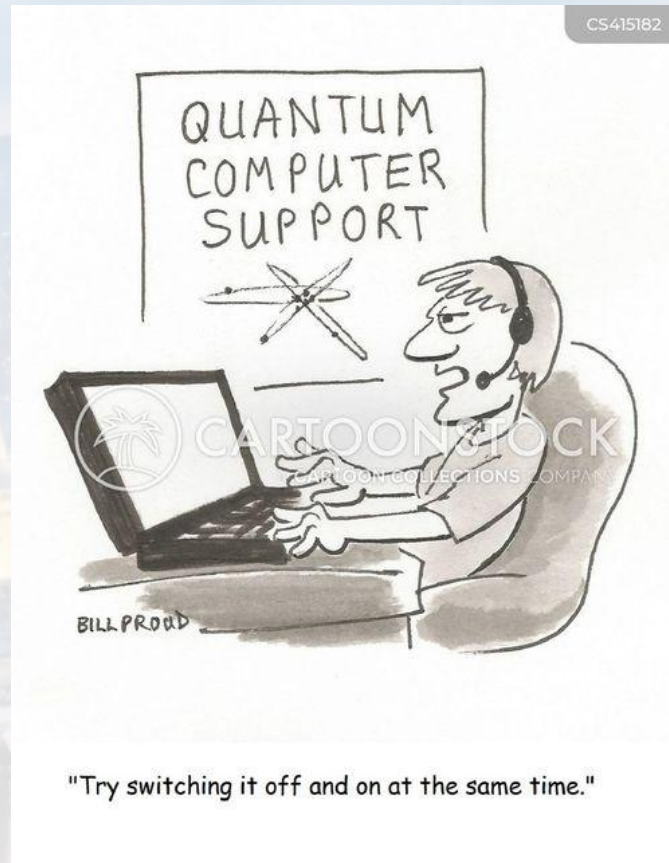


M. Hohle:

Physics 77: Introduction to Computational Techniques in Physics





Lecture: **Wed** 2:00 - 4:00pm

Workshops: WS1 **Fr** 2:00 - 4:00pm, Anthro/Art Practice Bldg 155

WS2 **Fr** 4:00 - 6:00pm, Social Sciences Building 170

Office Hours Lecturer: **Wed** 4:30 – 6:00pm

Instruction begins: Wednesday, August 26th, 2026

Instruction ends: Wednesday, December 2nd, 2026

check out: [Google Colab](#)

[bcourses](#)

[datahub.berkeley](#)

my contact: markus.hohle@berkeley.edu

Dingyi: dingyisun@berkeley.edu



PHYSICS 77/88

Introduction to Computational Techniques & Data Science Applications in Physics



Orientation



Syllabus



Course Map



Capstone Project



Support



Instructors (Physics)

Welcome!

Welcome to **Physics 77/88, Introduction to Computational Techniques and Data Science Application in Physics!**

This course will introduce you to scientific programming in Python with examples from physics. The course includes lectures, interactive workshops, homework assignments and a Final Capstone Project. A key philosophy in the course is that the best way to learn Python and programming is *by doing*. So let's get started with learning and applying these computational techniques in physics.

Helpful Links

1. Introduction to [Canvas platform](#) used for bCourse
2. Don't get lost: [Campus Map](#) Berkeley
3. Access [Zoom](#) via your CalID
4. Seek **help** when you are **depressed and stressed** [here](#)
5. You'd like to use **GitHub**, but don't want to run it via command line
6. Link to download **ANACONDA Navigator** or **miniconda**, the coding

Important Dates and Times (all times are in PST)

Lecture:

Start: Wednesday, August 26th

End: Wednesday, December 2nd

When: Wednesday, 2:00 - 4:00pm

Where: Etcheverry 3106

Workshops:

Workshop I: Friday, 2:00 - 4:00pm, Anthro/Art Practice Bldg 155

Workshop II: Friday, 4:00 - 6:00pm, Social Sciences Building 170

Office Hours:

When: Wednesday, 4:30 - 6:00pm

Where: TBD, Physics North



PHYSICS 77/88

Introduction to Computational Techniques & Data Science Applications in Physics



Syllabus

Orientation

Course Map

Capstone Project

Support

Instructors (Physics)

Welcome!

Welcome to **Physics 77/88, Introduction to Computational Techniques and Data Science Application in Physics!**

This course will introduce you to scientific programming in Python with examples from physics. The course includes lectures, interactive workshops, homework assignments and a Final Capstone Project. A key philosophy in the course is that the best way to learn Python and programming is *by doing*. So let's get started with learning and applying these computational techniques in physics.



Course Description

Learning Goals

Course Map

Instructor Information and Communication

Course Materials and Technical Requirements

Learning Activities and Assignments

Grading

Strategies for Successful Learning

Course Policies



Course Description

Learning Goals

Course Map

Instructor Information and Communication

Course Materials and Technical Requirements

Learning Activities and Assignments

Grading

Strategies for Successful Learning

Course Policies

Week	Dates	Topics	Reading	Material
1	Aug 26th	Programming Environment & UIs for Python, Programming Fundamentals	Setup Guidelines	Module 1
2	Sep 2nd	Basic Types in Python		Module 2
3	Sep 9th	Parsing, Data Processing and File I/O		Module 3
4	Sep 16th	Visualization: Matplotlib & Seaborn		Module 4
5	Sep 23rd	Functions, Map & Lambda		Module 5
6	Sep 30th	Random Numbers & Probability Distributions		Module 6
7	Oct 7th	Interpreting Measurements & Introduction to Bayesian Statistics	DS Sivia Data Analysis, Ch 2 & 3	Module 7
8	Oct 14th	Numerical Integration and Differentiation		Module 8
9	Oct 21st	Root finding, Interpolation		Module 9
10	Oct 28th	Systems of Linear Equations		Module 10
11	Nov 4th	Introduction to Ordinary Differential Equations (ODEs)		Module 11
	Nov 11th	<i>Veteran's Day</i>		
12	Nov 18th	Stability of ODEs, Runge-Kutta, Examples		Module 12
	Nov 25th	<i>Thanksgiving</i>		
13	Dec 2nd	Final Project Presentations		Module 13
	Dec 7th - 11th	RRR		

links to the material



Week	Date	Topic
1	Aug 26 th	Programming Environment & UIs for Python. Programming Fundamentals
2	Sep 2 nd	Basic Types in Python
3	Sep 9 th	Parsing, Data Processing and File I/O
4	Sep 16 th	Visualization: Matplotlib & Seaborn
5	Sep 23 rd	Functions, Map & Lambda
6	Sep 30 th	Random Numbers & Probability Distributions
7	Oct 7 th	Interpreting Measurements & Introduction to Bayesian Statistics
8	Oct 14 th	Numerical Integration and Differentiation
9	Oct 21 st	Root finding, Interpolation
10	Oct 28 th	Systems of Linear Equations
11	Nov 4 th	Introduction to Ordinary Differential Equations (ODEs)
	Nov 11 th	<i>Veteran's Day</i>
12	Nov 18 th	Stability of ODEs, Runge-Kutta, Examples
	Nov 25 th	<i>Thanksgiving</i>
13	Dec 2 nd	Final Project Presentations



Mixed Audience

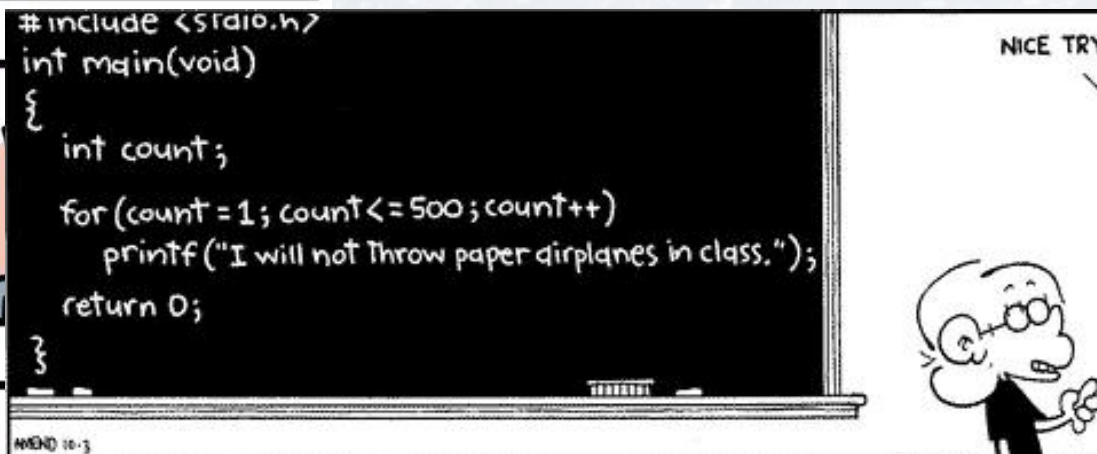
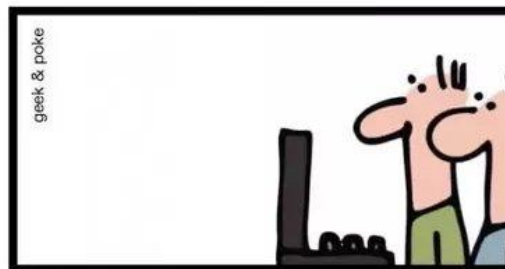
beginner



medium



advanced





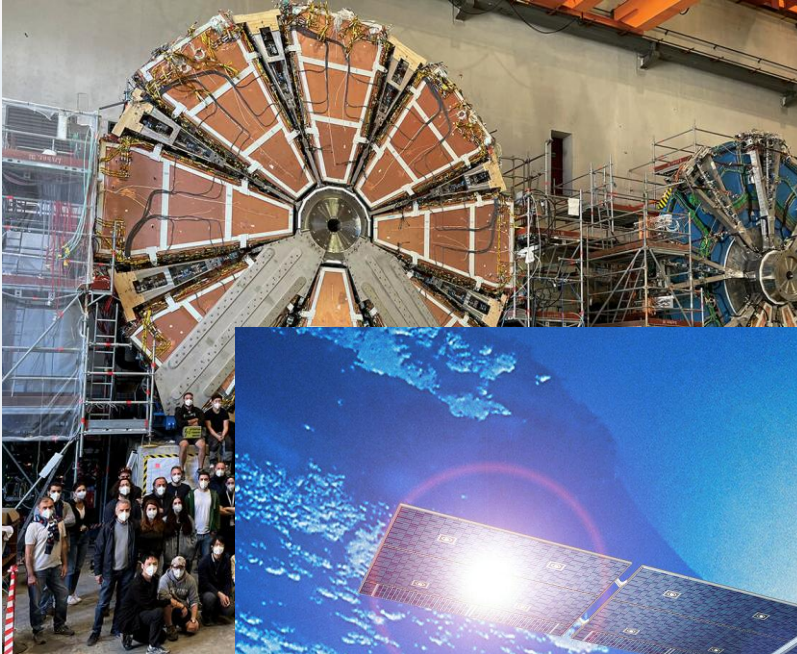
motivation: The good old days:



Salar de Uyuni,
Bolivia, Mar 2003



motivation:



source: CERN



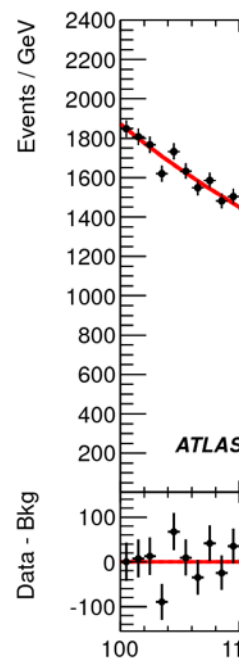
source: LIGO collaboration



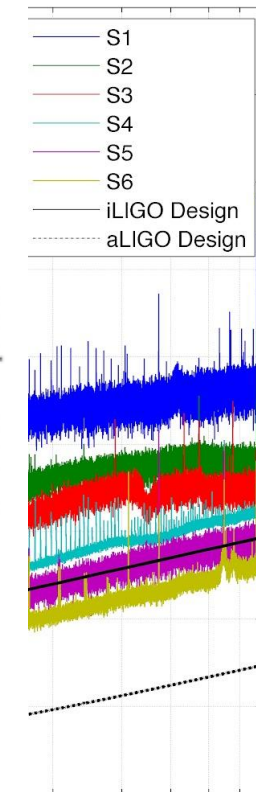
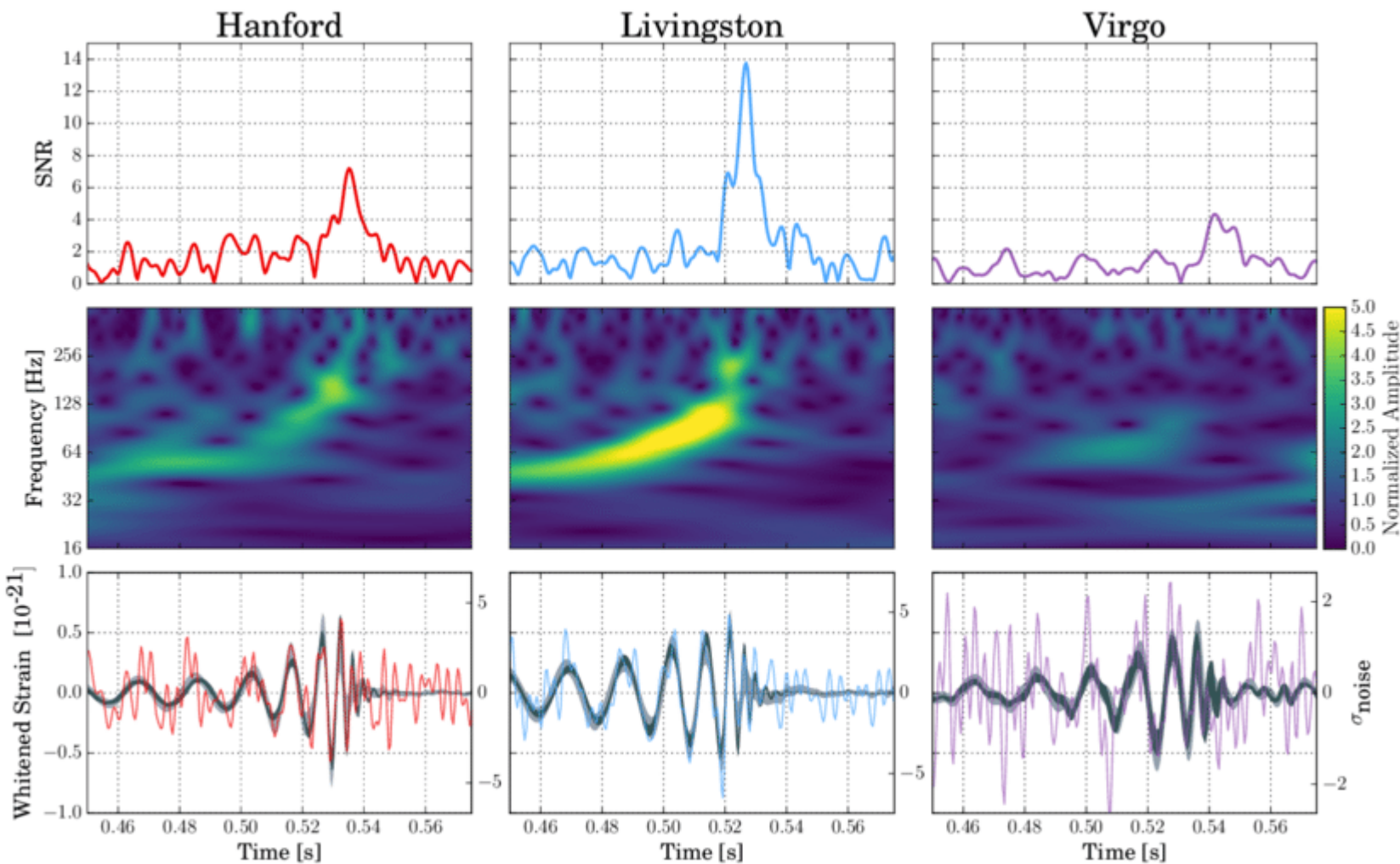
source: ESA



motivation:



Source: CERN/



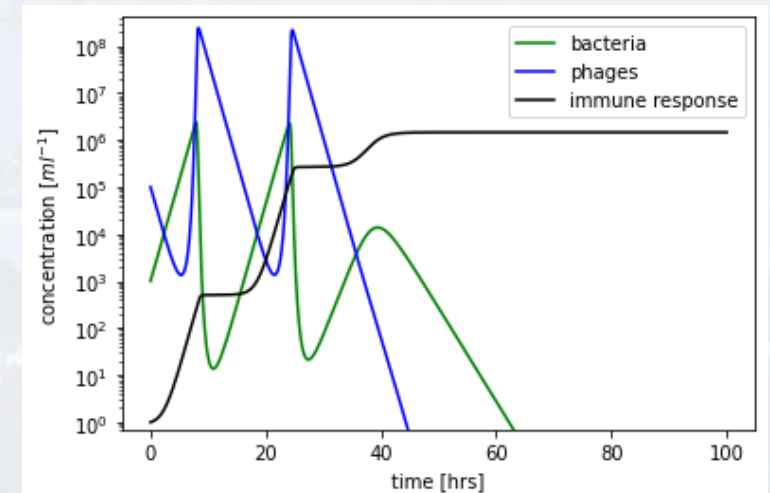
Source: Sergio Gaudio Embry-Riddle Aeronautical University, Prescott, United States · Physics



being able to write code like this...

```
@my_timer  
class SignalDetect():  
  
    def __init__(self, T, Range_phi = [0, 2*np.pi], dphi = 0.01,\  
                  MaxM = 20, **Opts):  
  
        L = listdir(getcwd())  
        [remove(i) for i in L if '.npy' in i]  
  
        ...
```

...and simulating systems like that





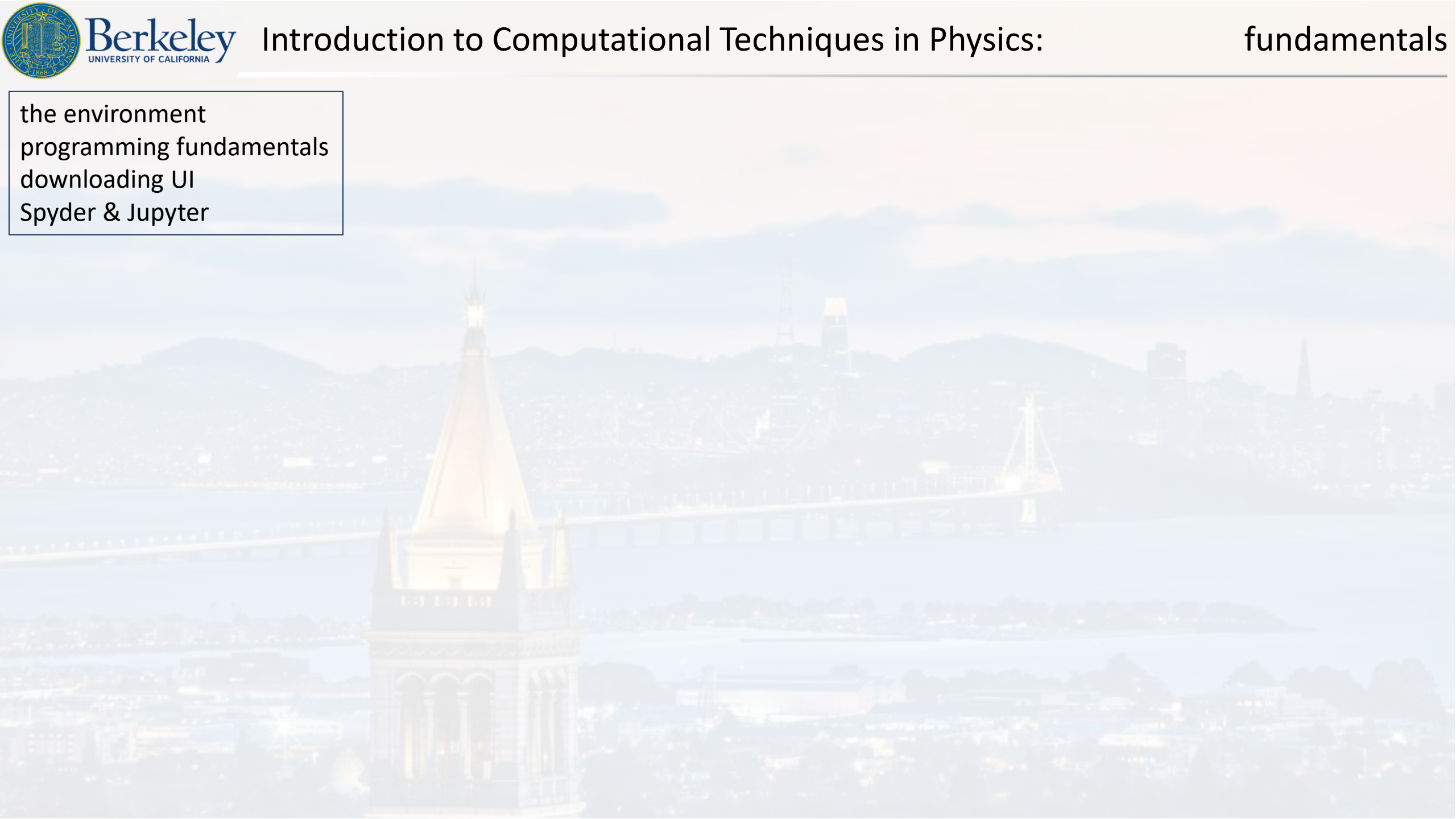
Week	Date	Topic
1	Aug 26 th	Programming Environment & UIs for Python. Programming Fundamentals
2	Sep 2 nd	Basic Types in Python
3	Sep 9 th	Parsing, Data Processing and File I/O
4	Sep 16 th	Visualization: Matplotlib & Seaborn
5	Sep 23 rd	Functions, Map & Lambda
6	Sep 30 th	Random Numbers & Probability Distributions
7	Oct 7 th	Interpreting Measurements & Introduction to Bayesian Statistics
8	Oct 14 th	Numerical Integration and Differentiation
9	Oct 21 st	Root finding, Interpolation
10	Oct 28 th	Systems of Linear Equations
11	Nov 4 th	Introduction to Ordinary Differential Equations (ODEs)
	Nov 11 th	<i>Veteran's Day</i>
12	Nov 18 th	Stability of ODEs, Runge-Kutta, Examples
	Nov 25 th	<i>Thanksgiving</i>
13	Dec 2 nd	Final Project Presentations



Week	Date	Topic
1	Aug 26 th	Programming Environment & UIs for Python. Programming Fundamentals
2	Sep 2 nd	Basic Types in Python
3	Sep 9 th	Parsing, Data Processing and File I/O
4	Sep 16 th	Visualization: Matplotlib & Seaborn
5	Sep 23 rd	Functions, Map & Lambda
6	Sep 30 th	Random Numbers & Probability Distributions
7	Oct 7 th	Interpreting Measurements & Introduction to Bayesian Statistics
8	Oct 14 th	Numerical Integration and Differentiation
9	Oct 21 st	Root finding, Interpolation
10	Oct 28 th	Systems of Linear Equations
11	Nov 4 th	Introduction to Ordinary Differential Equations (ODEs)
	Nov 11 th	<i>Veteran's Day</i>
12	Nov 18 th	Stability of ODEs, Runge-Kutta, Examples
	Nov 25 th	<i>Thanksgiving</i>
13	Dec 2 nd	Final Project Presentations



the environment
programming fundamentals
downloading UI
Spyder & Jupyter





the environment

programming fundamentals

downloading UI

Spyder & Jupyter

before we can start coding...we need an **environment**

your computer

Operational System (OS)

interface between the computer and all programs,
usually, Windows or Linux



the environment

programming fundamentals
downloading UI
Spyder & Jupyter

before we can start coding...we need an **environment**

your computer

Operational System (OS)

interface between the computer and all programs,
usually Windows or **Linux**

- in Computer Science, Software Engineering, Physics etc:
some version of Linux/Unix
- for Windows:

Windows Subsystem for Linux (WSL), see later





the environment

programming fundamentals
downloading UI
Spyder & Jupyter

before we can start coding...we need an **environment**

your computer

Operational System (OS)

Linux/WSL

Text Editor

- for actual coding
- in principle: any editor is fine

- but: code needs to be **compiled** and **ran**!



the environment

programming fundamentals
downloading UI
Spyder & Jupyter

before we can start coding...we need an **environment**

your computer

Operational System (OS)

Linux/WSL

Coding platform

- **ANACONDA** (editor + compiler: **Jupyter**, **Spyder** + many other tools)
- **miniconda** (like **ANACONDA** but only basics, editor + compiler: **Jupyter**)
- **VS Code** (editor + compiler)

Text Editor

- for actual coding
- in principle: any editor is fine



the environment

programming fundamentals
downloading UI
Spyder & Jupyter

VS Code
(professionals)

before we can start coding...we need an **environment**

```
serviceWorker.js — create-react-app

src > JS serviceWorker.js
28 // Our service worker won't work if PUBLIC_URL is on a different origin
29 // from what our page is served on. This might happen if a CDN is used
30 // to serve assets; see https://github.com/facebook/create-react-app/issues/
31 // 5016
32 return;
33
34 window.addEventListener('load', () => {
35   const { addEventListener } = window;
36   if (isApplicationCache) {
37     // Try to see if this works first (async)
38     // See https://github.com/facebook/create-react-app/issues/5016
39     checkForUpdates();
40     // AbortController
41     // AbortSignal
42     // AbstractRange
43     navigator.userAgent;
44     // AnalyserNode
45     // Animation
46     // AnimationEffect
47   });
48 } else {
49   // Is not localhost. Just register service worker
50   registerValidSW(swUrl, config);
51 }
52
53 }

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
node
Compiled successfully!
You can now view create-react-app in the browser.

Local: http://localhost:3000/
On Your Network: http://192.168.86.138:3000/

Note that the development build is not optimized.
To create a production build, use yarn build.
```

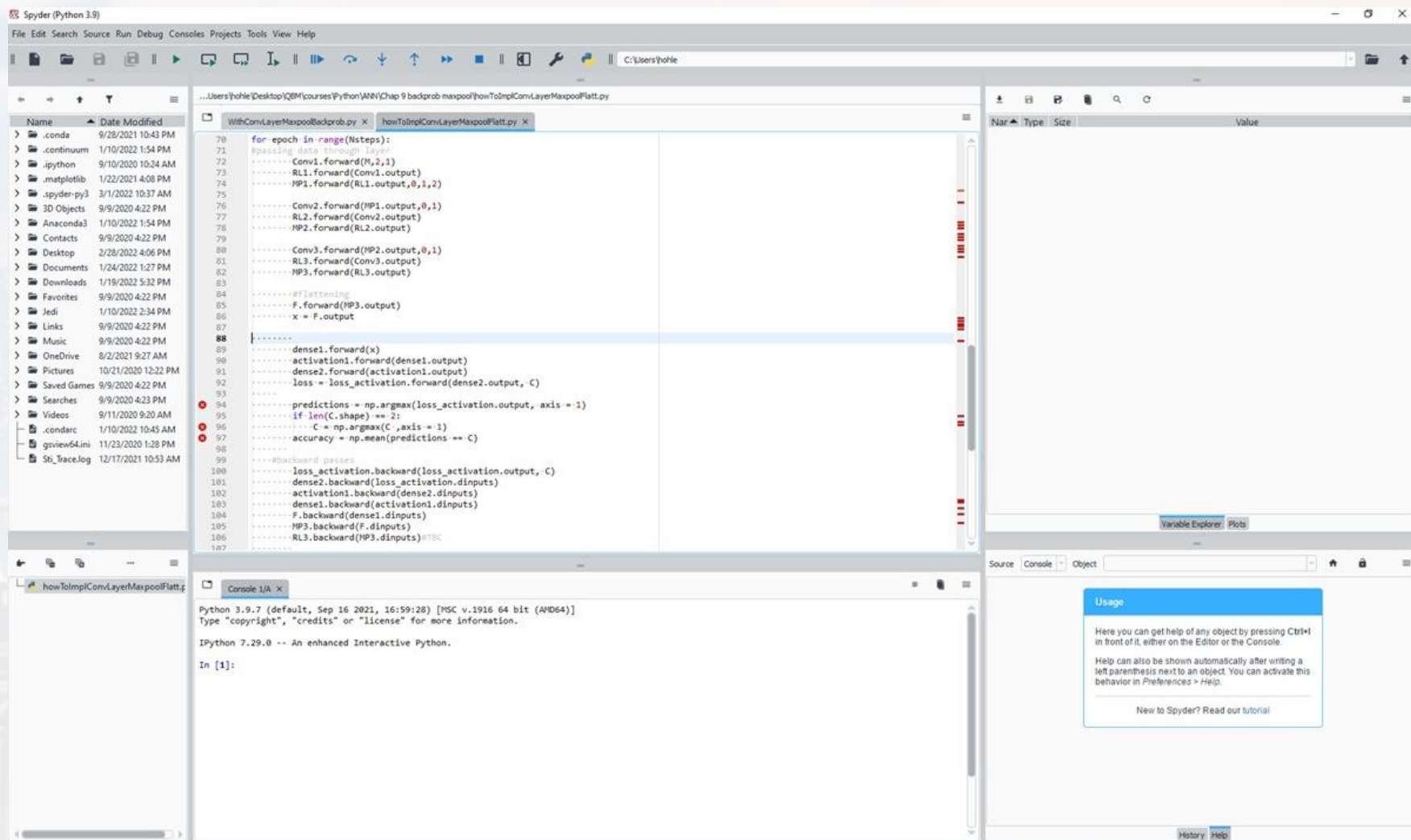


the environment

programming fundamentals
downloading UI
Spyder & Jupyter

before we can start coding...we need an **environment**

Spyder (see later)





the environment

programming fundamentals
downloading UI
Spyder & Jupyter

before we can start coding...we need an **environment**

Jupyter (see later)

```
jupyter ANNIII Last Checkpoint: 3 months ago
File Edit View Run Kernel Settings Help
+ ✂ 📄 📌 ▶ ■ ↺ ⏩ Markdown ▾

1) Defining all the Parts we need for an ANN

We start with defining the dense layer, including backpropagation:

[7]: class Layer_Dense():

    def __init__(self, n_inputs, n_neurons):
        self.weights = np.random.randn(n_inputs, n_neurons)
        self.biases = np.zeros((1, n_neurons))

    def forward(self, inputs):
        self.output = np.dot(inputs, self.weights) + self.biases
        self.inputs = inputs

    def backward(self, dvalues):
        #gradients
        self.dweights = np.dot(self.inputs.T, dvalues)
        self.dbiases = np.sum(dvalues, axis = 0, keepdims = True)
        self.dinputs = np.dot(dvalues, self.weights.T)

and as well as for the activation layer.

[11]: class Activation_ReLU():
```



the environment

programming fundamentals
downloading UI
Spyder & Jupyter

before we can start coding...we need an **environment**

your computer

Operational System (OS)

Linux/WSL

Coding platform

finetuning settings (ANACONDA or miniconda **environment**)

- **ANACONDA** (editor + compiler: **Jupyter**, **Spyder** + many other tools)
- **miniconda** (like **ANACONDA** but only basics, editor + compiler: **Jupyter**)

- **VS Code** (editor + compiler)

Text Editor

- for actual coding
- in principle: any editor is fine



the environment

programming fundamentals

downloading UI

Spyder & Jupyter



created 1990, Guido van Rossum

named after “**Monty Python**”, not the serpent



idea: flexible, simple and compact syntax, extendable



the environment

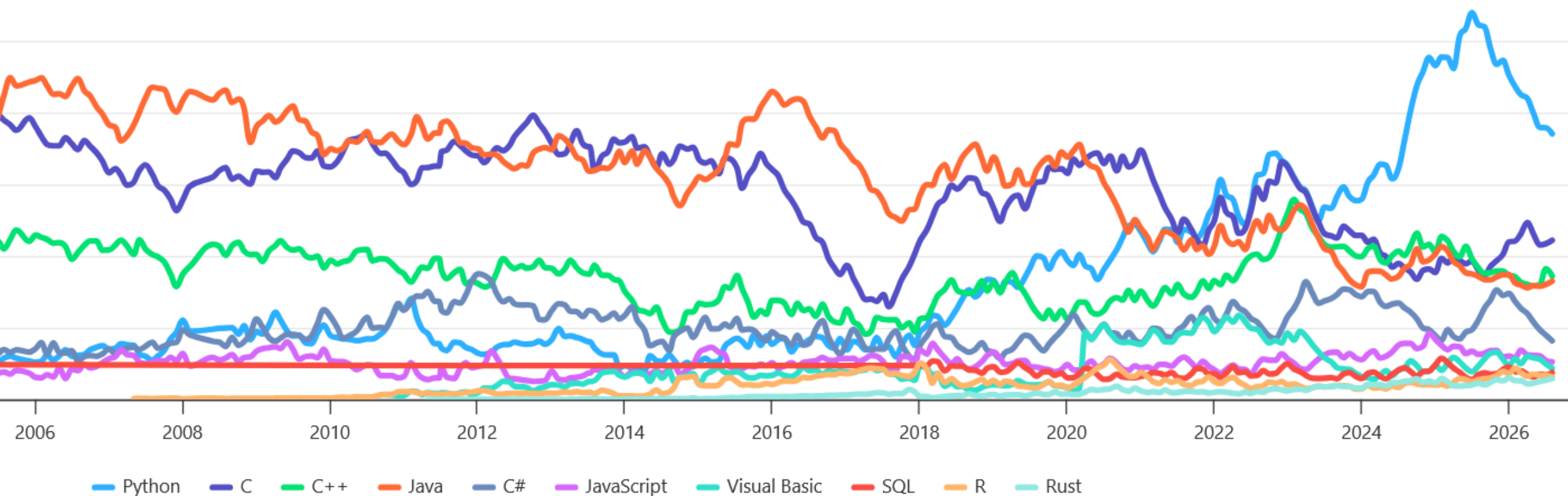
programming fundamentals

downloading UI

Spyder & Jupyter

TIOBE index Aug 2026

Programming Language		Ratings	Change
	Python		18.53%
	C	11.10%	+2.07%
	C++	8.62%	-0.56%
	Java	8.05%	-0.41%





Programming Languages: translate human instructions to a form understandable by a computer

procedural:

functions/ routines that call each other
(Fortran, ALGOL, COBOL, BASIC, Pascal, C)

the “*style*”
of programming

object oriented (OOP):

creating objects/types of different properties (see later)
C++, Fortran 2003, Java, MATLAB, **Python**, Ruby, ...

compiled language:

close to the resulting machine code, **fast**
(mainly because of memory allocation management)
Fortran, C, C++, Java, Cobol, Pascal

how a programming
language “*talks*” to your
CPU or GPU

interpreted language:

an interpreter translates between source code and
machine code. **Slower, but simpler syntax**
Perl, Raku, **Python**, MATLAB



Python libraries/modules

```
from my_module import my_method as my_alias
```

1) reading files (.xlsx, .xls, .csv, .txt, ...)

pandas (standard), dask, polars

2) plotting

matplotlib, seaborn

3) numerical methods

math, numpy, scipy

4) machine learning

scikitlearn

5) ANN/AI/DeepLearning



Keras





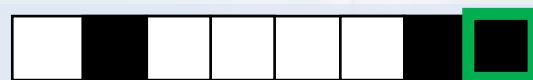
the environment

programming fundamentals

downloading UI

Spyder & Jupyter

We need only **two** states: **on** and **off**



$n = 8$

smallest memory cell: **8bits = 1byte**

one bit (**b**inary **d**igit)

yes/no

on/off

up/down

How many different states can I create with 8 switches? $2 \times 2 \times 2 \dots = 2^n$

8 bit :	$2^8 =$	256
16 bit:	$2^{16} =$	65.536
32 bit:	$2^{32} =$	4.294.967.296
64 bit:	$2^{64} =$	18.446.744.073.709.551.616



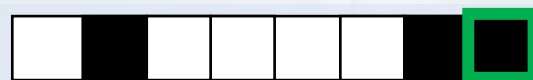
the environment

programming fundamentals

downloading UI

Spyder & Jupyter

We need only **two** states: **on** and **off**



$n = 8$

smallest memory cell: **8bits = 1byte**

8 bit :	$2^8 =$	256
16 bit:	$2^{16} =$	65.536
32 bit:	$2^{32} =$	4.294.967.296
64 bit:	$2^{64} =$	18.446.744.073.709.551.616

0	00000000
1	0000000 1
2	000000 10
3	000000 11
4	00000 100
5	00000 101
6	00000 110
7	00000 111
8	0000 1000
9	0000 1001
10	0000 1010
11	0000 1011
12	0000 1100
13	0000 1101
14	0000 1110
15	0000 1111
16	000 10000
...	



the environment

programming fundamentals

downloading UI

Spyder & Jupyter

We need only **two** states: **on** and **off**

$$0 = 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$$

$$1 = 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

$$2 = 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$

$$7 = 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

8 bit :	$2^8 =$	256
16 bit:	$2^{16} =$	65.536
32 bit:	$2^{32} =$	4.294.967.296
64 bit:	$2^{64} =$	18.446.744.073.709.551.616

0	00000000
1	0000000 1
2	000000 10
3	000000 11
4	00000 100
5	00000 101
6	00000 110
7	00000 111
8	0000 1000
9	0000 1001
10	0000 1010
11	0000 1011
12	0000 1100
13	0000 1101
14	0000 1110
15	0000 1111
16	000 10000
...	

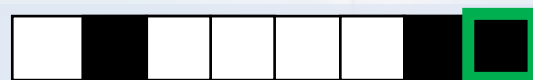


the environment

programming fundamentals

downloading UI

Spyder & Jupyter



$n = 8$

smallest memory cell: **8bits = 1byte**

8 bit :	$2^8 =$	256
16 bit:	$2^{16} =$	65.536
32 bit:	$2^{32} =$	4.294.967.296
64 bit:	$2^{64} =$	18.446.744.073.709.551.616

8 bits = 1 byte (B)

1 kB = 1024 B

1 MB = 1024 kB etc

accuracy of numerical operations:

```
import sys
sys.float_info.epsilon
```

machine epsilon

2.220446049250313e-16



the environment

programming fundamentals

downloading UI

Spyder & Jupyter

	rel. approx. error (ϵ)	range
16 bit int		-32768 ... 32767
32 bit int		$\approx -10^9 \dots 10^9$
32 bit float	$\approx 10^{-8}$	$\approx 10^{-38} \dots 10^{38}$
64 bit double	$\approx 10^{-16}$	$\approx 10^{-308} \dots 10^{308}$

0 00000000
1 0000000**1**
2 000000**10**
3 000000**11**
4 00000**100**
5 00000**101**
6 00000**110**
7 00000**111**
8 0000**1000**
9 0000**1001**
10 0000**1010**
11 0000**1011**
12 0000**1100**
13 0000**1101**
14 0000**1110**
15 0000**1111**
16 000**10000**
...

Luckily, Python will tell us when an operation doesn't make sense based on precision, but still: **be cautious!**



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

beginner



ANACONDA Navigator

- straight forward installation
- free for teaching/instructions
- more tools than we actually need
- your choice if you have never programmed

medium



miniconda

- requires a few more steps to install
- free
- you need to install the specific tools
- your choice if you prefer flexibility over convenience



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

ANACONDA Navigator

- straight forward installation
- free for teaching/instructions
- more tools than we actually need
- your choice if you have never programmed

download [here](#)

Distribution Installers

 **Download**

For installation assistance, refer to [troubleshooting](#).



Windows



Mac



Linux





the environment
programming fundamentals
downloading UI
Spyder & Jupyter

ANACONDA Navigator

- straight forward installation
- free for teaching/instructions
- more tools than we actually need

The only Python IDE you need – built for data and AI/ML professionals. Supercharged with an AI-enhanced IDE experience. Free forever, plus one month of Pro included. Install	Access various large language models (LLMs) curated by Anaconda, and start leveraging secure local AI today. Install	4.0.15 Anaconda Assistant JupyterLab supercharged with a suite of Anaconda extensions, starting with the Anaconda Assistant AI chatbot. Launch	Cloud-hosted notebook service from Anaconda. Launch a preconfigured environment with hundreds of packages and store project files with persistent cloud storage. Launch	Run Python Jupyter notebooks on a multi-model database. Launch	1.0.0 Opens a terminal instance with conda activated (requires menuinst 2.1.1 or greater). Launch
 CMD.exe Prompt 0.1.1 Run a cmd.exe terminal with your current environment from Navigator activated Launch	 console_shortcut_miniconda 0.1.1 Anaconda Powershell Prompt Launch	 JupyterLab 4.0.11 An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture. Launch	 Notebook 7.0.8 Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis. Launch	 Powershell Prompt 0.0.1 Run a Powershell terminal with your current environment from Navigator activated Launch	 Qt Console 5.6.1 PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more. Launch
 Spyder 6.0.5 Scientific Python Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features Launch	 VS Code 1.100.2 Streamlined code editor with support for development operations like debugging, task running and version control. Launch	 EduBlocks Web-based coding platform from Anaconda designed for students. Learn Python coding through an interactive, block-based visual environment. Launch	 IBM watsonx IBM watsonx is an enterprise-ready AI platform including a data store, model builder, and AI model management and monitoring. Launch	 Oracle Data Science Service OCI Data Science offers a machine learning platform to build, train, manage, and deploy your machine learning models on the cloud with your favorite open-source tools Launch	 PyScript Code and share Python in the Browser. A vibrant community of makers, builders, and hackers building the next frontier of Python-powered web applications. Launch



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda

- requires a few more steps to install
- free
- you need to install the specific tools
- your choice if you prefer flexibility over convenience



on PC: install **W**indows **S**ubsystem for **L**inux

follow the instructions [here](#)

watch [1a_WSL.mp4](#) for PC
[1b_Mac.mp4](#) for Mac

The screenshot shows the Microsoft Learn website. The main heading is "How to install Linux on Windows with WSL". Below the heading, it says "Article • 11/19/2024 • 10 contributors". There is a sidebar on the left with a search bar and a list of links: "WSL Documentation", "Overview", "Install" (with "Install WSL" selected), "Manual install steps for older versions", "Install on Windows Server", "Tutorials", "Concepts", "How-to", and "Download PDF". On the right, there are sections for "Additional resources" including "Training" (with a link to "Developing in the Windows Subsystem for Linux with Visual Studio Code - Training") and "Certification" (with a link to "Microsoft Certified: Windows Server Hybrid Administrator Associate - Certifications").



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda on PC:



open your **WSL** shell and run the following commands:

1) download installer:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

```
$ wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

2) run installer:

```
bash Miniconda3-latest-Linux-x86_64.sh
```

confirm - license agreement,
- folder location and
- settings

3) close and reopen **WSL**

watch [2_Miniconda.mp4](#)



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda on PC:

5) create environment: `conda create --name <My_Environment>`

6) check environment: `conda env list`

7) activate environment: `conda activate <My_Environment>`

`<My_Environment>`

```
(base) mmh@DESKTOP-PPSA666:/mnt/c/Users/MMH_user$ conda activate physics-python  
(physics-python) mmh@DESKTOP-PPSA666:/mnt/c/Users/MMH_user$ |
```

watch [3_PhysEnv.mp4](#)



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda on PC:



9a) install Jupyterlab:

```
conda install -c conda-forge jupyterlab
```

confirm settings

**9b) install Jupyter notebook
and Spyder:**

```
conda install -c conda-forge ipykernel spyder-notebook jupyter
```

**9c) install scientific computing
packages:**

```
conda install -c conda-forge numpy scipy scikit-learn
```

**9d) install plotting tools
and reader:**

```
conda install -c conda-forge pandas matplotlib seaborn
```

watch [4_PythonPackages.mp4](#)



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda on PC:



10a) open Jupyter:

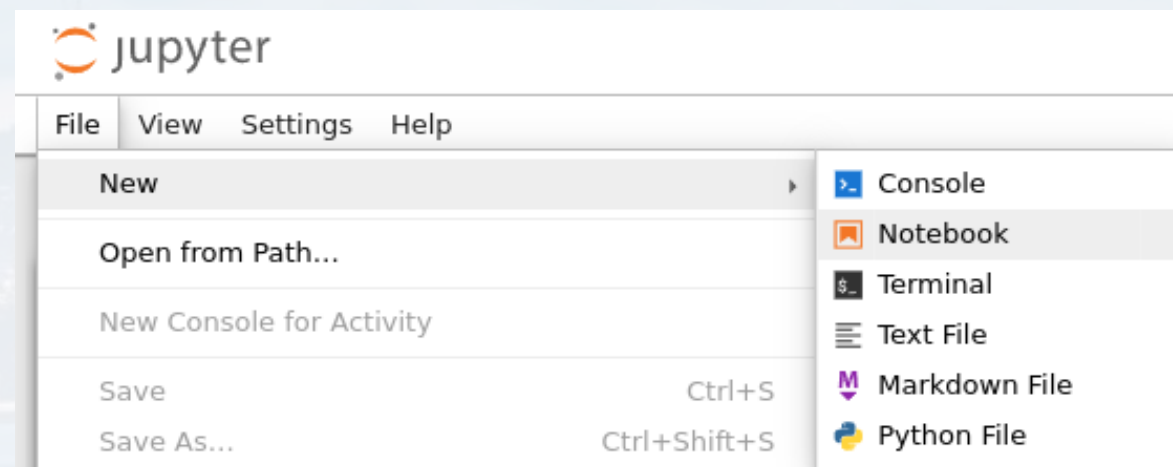
jupyter notebook &

a browser will open

go to → File
→ New
→ Notebook
→ confirm kernel

the notebook will open

watch [5_TestingUIs.mp4](#)



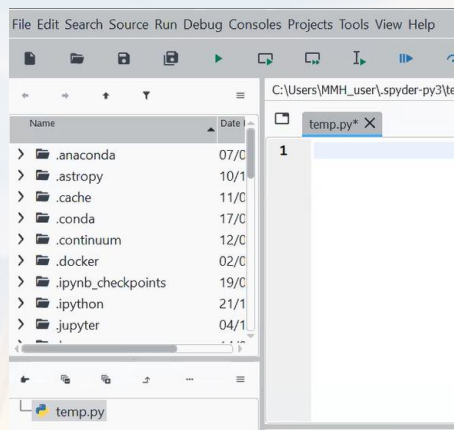


the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda on PC:

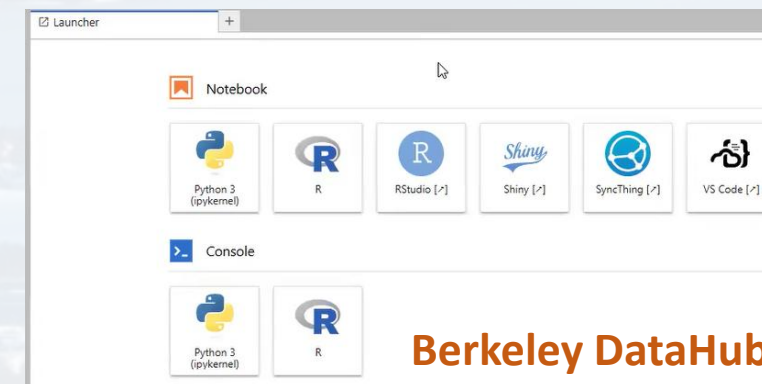
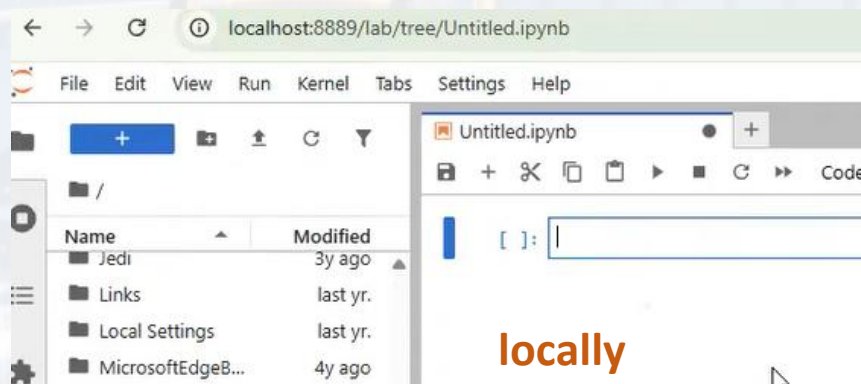
10b) open Spyder:

`spyder &`



10c) open Jupyter lab:

`jupyter lab &`





the environment
programming fundamentals
downloading UI
Spyder & Jupyter

miniconda on MAC:

follow instructions [here](#)



Installing Miniconda and compilers

Click the appropriate tab for your operating system to see set-up instructions.

Mac OS

Linux

Windows

Miniconda Installation

You can download and run the installer at this [link](#).

Compilers

MacOS users should [install XCode](#). An easy way to install XCode is through the [Mac App Store](#).



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

watch [7_Quick_Guide_Jupyter.mp4](#)
[8_Quick_Guide_Spyder.mp4](#)

The only Python IDE you need – built for data and AI/ML professionals. Supercharged with an AI-enhanced IDE experience. Free forever, plus one month of Pro included. Install	Access various large language models (LLMs) curated by Anaconda, and start leveraging secure local AI today. Install	4.0.15 Anaconda Assistant JupyterLab supercharged with a suite of Anaconda extensions, starting with the Anaconda Assistant AI chatbot. Launch	Cloud-hosted notebook service from Anaconda. Launch a preconfigured environment with hundreds of packages and store project files with persistent cloud storage. Launch	Run Python Jupyter notebooks on a multi-model database. Launch	1.0.0 Opens a terminal instance with conda activated (requires menuinst 2.1.1 or greater). Launch
 CMD.exe Prompt 0.1.1 Run a cmd.exe terminal with your current environment from Navigator activated Launch	 console_shortcut_miniconda 0.1.1 Anaconda Powershell Prompt Launch	 JupyterLab 4.0.11 An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture. Launch	 Jupyter Notebook 7.0.8 Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis. Launch	 Powershell Prompt 0.0.1 Run a Powershell terminal with your current environment from Navigator activated Launch	 Qt Console 5.6.1 PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more. Launch
 Spyder 6.0.5 Scientific Python Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features Launch	 VS Code 1.100.2 Streamlined code editor with support for development operations like debugging, task running and version control. Launch	 EduBlocks Web-based coding platform from Anaconda designed for students. Learn Python coding through an interactive, block-based visual environment. Launch	 IBM watsonx IBM watsonx is an enterprise-ready AI platform including a data store, model builder, and AI model management and monitoring. Launch	 Oracle Cloud Infrastructure Oracle Data Science Service OCI Data Science offers a machine learning platform to build, train, manage, and deploy your machine learning models on the cloud with your favorite open-source tools Launch	 PyScript Code and share Python in the Browser. A vibrant community of makers, builders, and hackers building the next frontier of Python-powered web applications. Launch



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

lofi hip hop radio - beats to
PLAYING

Home Page - Select or create a

localhost:8888/tree

jupyter

Quit Logout

Files Running Clusters

Select items to perform actions on them.

0 /

3D Objects

anaconda3

Contacts

Desktop

Documents

Downloads

Favorites

Jedi

Links

Music

OneDrive

Pictures

Saved Games

Searches

localhost:8888/tree#

Upload New

Notebook:

Python 3 (ipykernel)

Create a new notebook with Python 3 (ipykernel)

Text File

Folder

Terminal

2 months ago

5 days ago

a year ago

a year ago

a year ago

a year ago

a year ago

a year ago

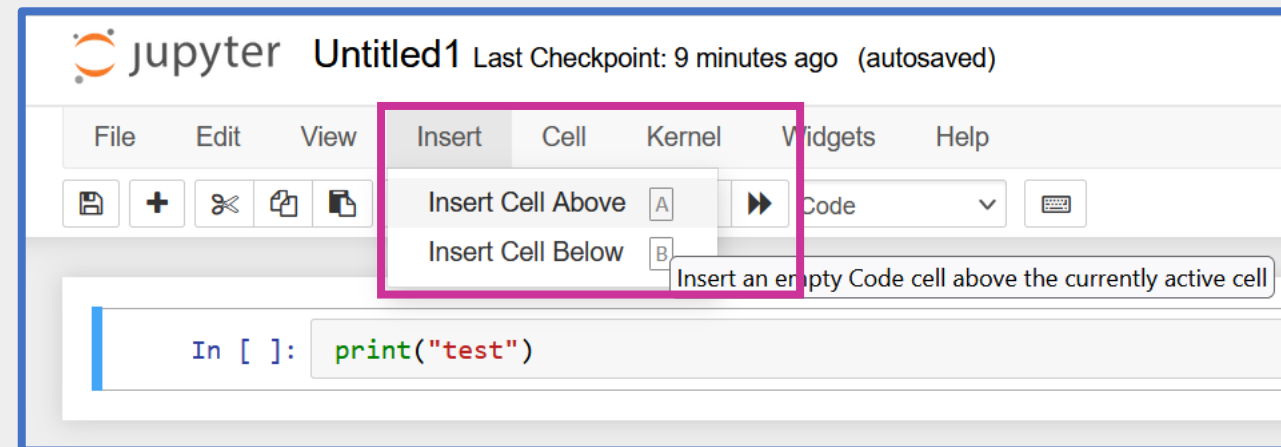
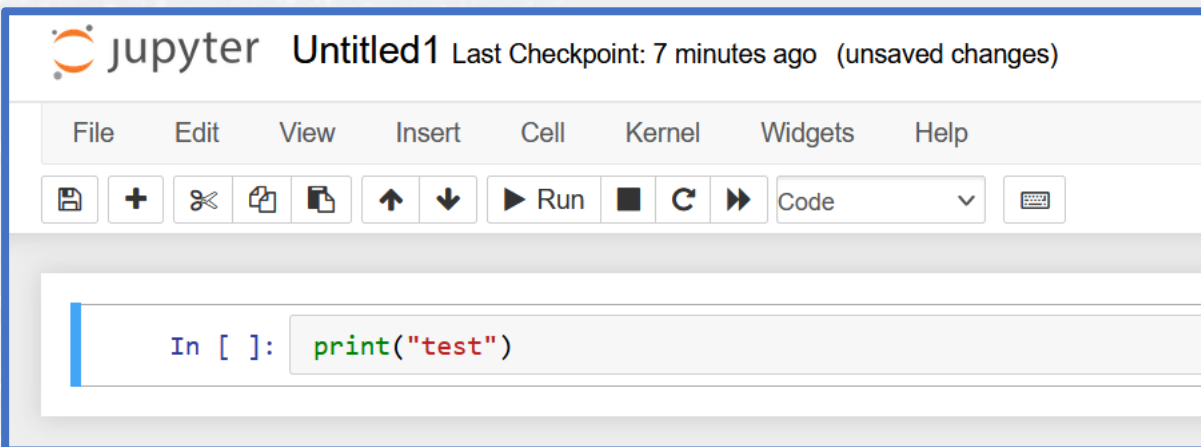
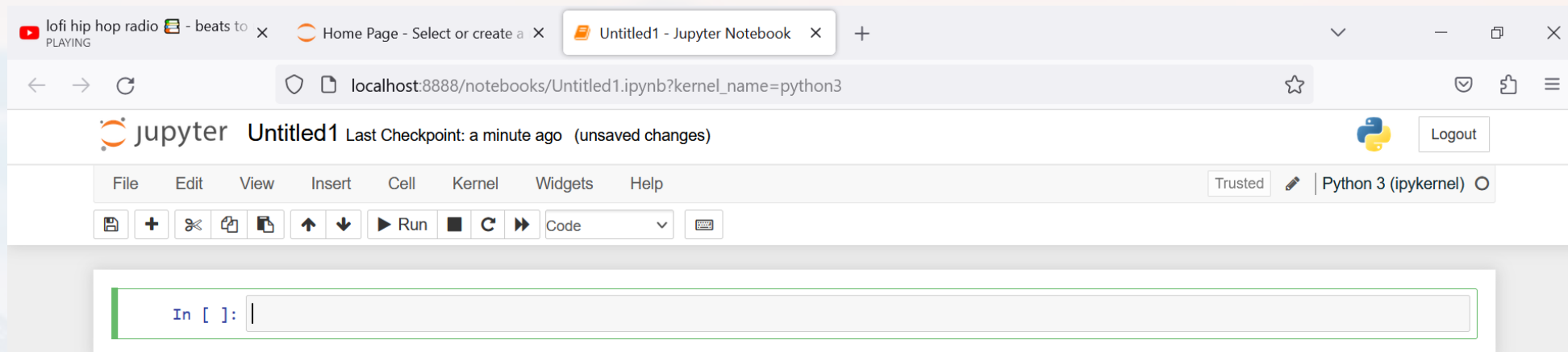
a year ago

Type here to search

11:33
04/12/2023

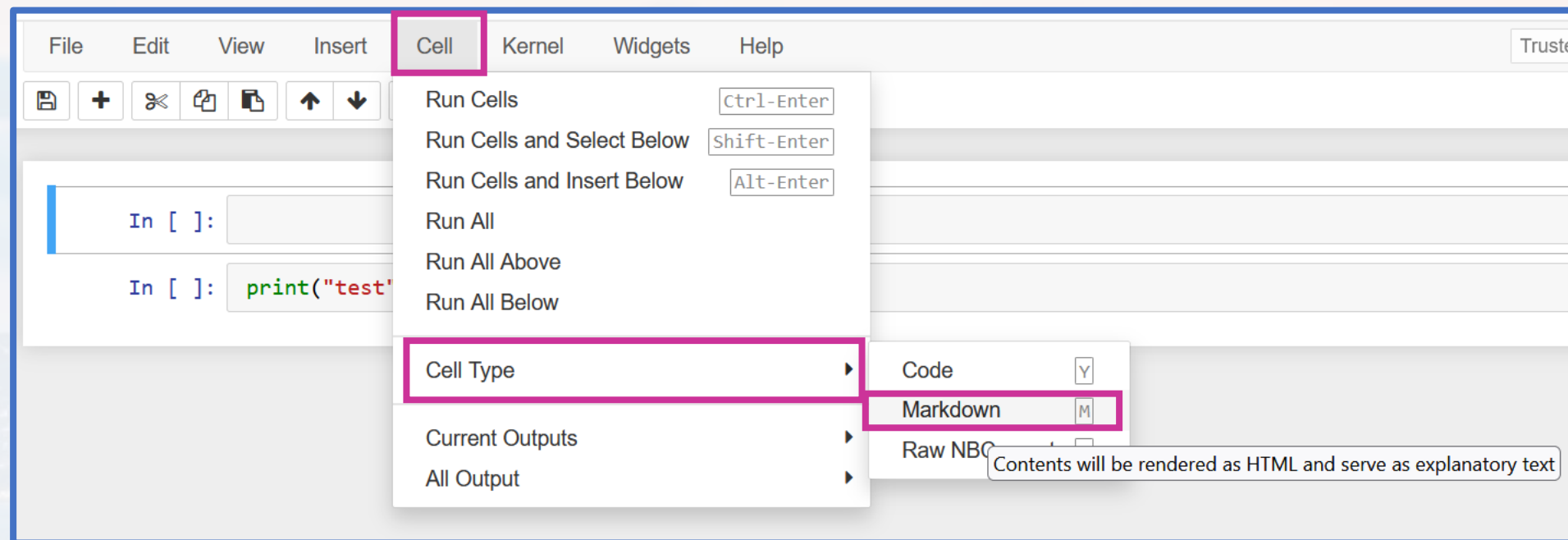


the environment
programming fundamentals
downloading UI
Spyder & Jupyter

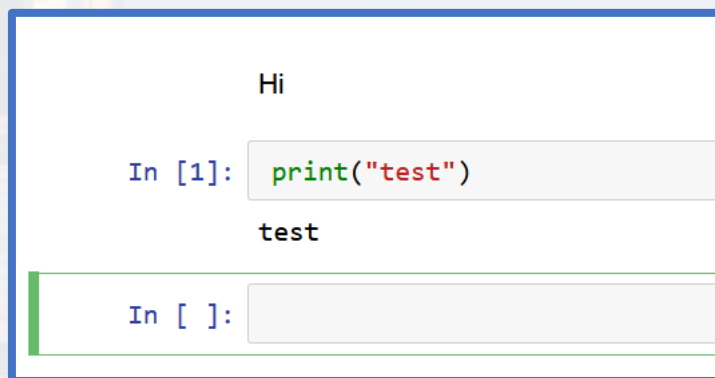




the environment
programming fundamentals
downloading UI
Spyder & Jupyter



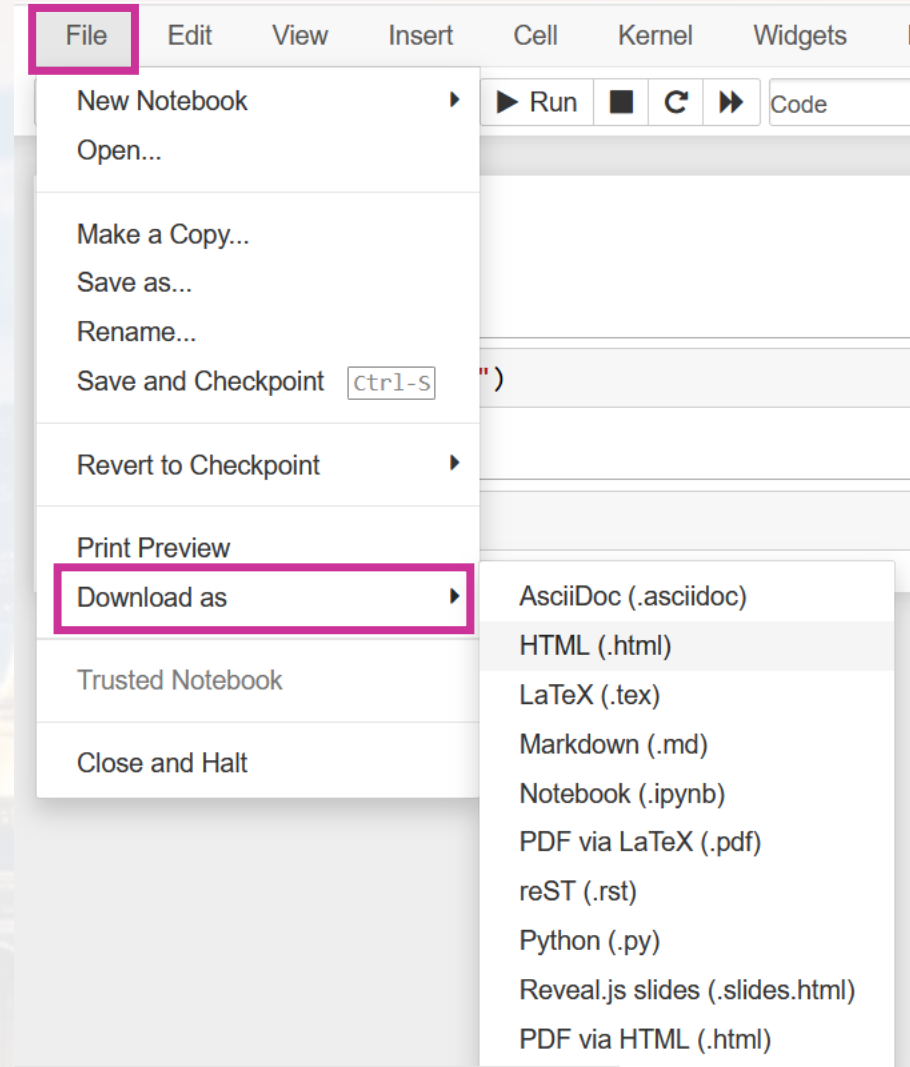
type something → click **run**



**web: Jupyter → markdown styles
(including LaTeX)**



the environment
programming fundamentals
downloading UI
Spyder & Jupyter





the environment
programming fundamentals
downloading UI
Spyder & Jupyter

You can run Jupyter on [Datahub](#) too!

Berkeley
UNIVERSITY OF CALIFORNIA

Operated by the [College of Computing, Data Science, and Society](#)

Log in to continue

Welcome to the University of California, Berkeley
DataHub.

 **jupyter**

 **Studio**

Berkeley
UNIVERSITY OF CALIFORNIA

CalNet Authentication Service

sign in

CalNet ID:

Passphrase (Case Sensitive):

SIGN IN

[HELP](#)

[Sponsored Guest Sign In](#)

[FORGOT CALNET ID OR PASSPHRASE?](#)

[MANAGE MY CALNET ACCOUNT](#)



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

You can run Jupyter on [Datahub](#) too!



DataHub

DataHub is requesting access to your account.

You are authorizing this app as [Markus Hohle](#).
Your email address is markus.hohle@berkeley.edu.

Cancel

Authorize



the environment
programming fundamentals
downloading UI

Spyder & Jupyter

You can run Jupyter on [Datahub](#) too!

Select Kernel

Select kernel for "Untitled.ipynb"

Python 3 (ipykernel) ▼

☐ Always start the preferred kernel

No Kernel

Select

Tabs Settings Help

Untitled.ipynb +

⌨ + ✂ 📄 📌 ▶ ■ ↺ ⏩ ⬇️ Download Format SQL ⌚ git Code ▼

[]:



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

watch [7_Quick_Guide_Jupyter.mp4](#)
[8_Quick_Guide_Spyder.mp4](#)

The only Python IDE you need – built for data and AI/ML professionals. Supercharged with an AI-enhanced IDE experience. Free forever, plus one month of Pro included. Install	Access various large language models (LLMs) curated by Anaconda, and start leveraging secure local AI today. Install	4.0.15 Anaconda Assistant JupyterLab supercharged with a suite of Anaconda extensions, starting with the Anaconda Assistant AI chatbot. Launch	Cloud-hosted notebook service from Anaconda. Launch a preconfigured environment with hundreds of packages and store project files with persistent cloud storage. Launch	Run Python Jupyter notebooks on a multi-model database. Launch	1.0.0 Opens a terminal instance with conda activated (requires menuinst 2.1.1 or greater). Launch
 CMD.exe Prompt 0.1.1 Run a cmd.exe terminal with your current environment from Navigator activated Launch	 console_shortcut_miniconda 0.1.1 Anaconda Powershell Prompt Launch	 JupyterLab 4.0.11 An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture. Launch	 Notebook 7.0.8 Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis. Launch	 Powershell Prompt 0.0.1 Run a Powershell terminal with your current environment from Navigator activated Launch	 Qt Console 5.6.1 PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more. Launch
 Spyder 6.0.5 Scientific Python Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features Launch	 VS Code 1.100.2 Streamlined code editor with support for development operations like debugging, task running and version control. Launch	 EduBlocks Web-based coding platform from Anaconda designed for students. Learn Python coding through an interactive, block-based visual environment. Launch	 IBM watsonx IBM watsonx is an enterprise-ready AI platform including a data store, model builder, and AI model management and monitoring. Launch	 Oracle Cloud Infrastructure Oracle Data Science Service OCI Data Science offers a machine learning platform to build, train, manage, and deploy your machine learning models on the cloud with your favorite open-source tools Launch	 PyScript Code and share Python in the Browser. A vibrant community of makers, builders, and hackers building the next frontier of Python-powered web applications. Launch



the environment
programming fundamentals
downloading UI
Spyder & Jupyter

**content of
current
folder**

The screenshot shows the Spyder Python IDE interface with several components highlighted by green boxes:

- folder navigator**: Located at the top right, it shows the file explorer for the current project.
- py script: for coding**: Located in the center, it shows the Python script being edited.
- workspace: displays current plots or variables**: Located on the right side, it shows the current workspace with variables and plots.
- console: typing commands & executing scripts**: Located at the bottom, it shows the IPython console where commands are typed and executed.

Additional details visible in the interface include:

- File Explorer (Left Panel)**: Shows the file structure of the current project, including folders like .conda, .continuum, .python, .matplotlib, .spyder-py3, 3D Objects, Anaconda3, Contacts, Desktop, Documents, Downloads, Favorites, Jedi, Links, Music, OneDrive, Pictures, Saved Games, Searches, Videos, .condarc, gsview64.ini, and Stl_Trace.log.
- Code Editor (Center Panel)**: Displays a Python script for a neural network. The script includes imports for numpy, keras, and matplotlib, and defines a function for training a neural network. The code is as follows:

```
70 for epoch in range(Nsteps):
71     #passing data through layer
72     -----Conv1.forward(M,2,1)
73     -----RL1.forward(Conv1.output)
74     -----MP1.forward(RL1.output,0,1,2)
75
76     -----Conv2.forward(MP1.output,0,1)
77     -----RL2.forward(Conv2.output)
78     -----MP2.forward(RL2.output)
79
80     -----Conv3.forward(MP2.output,0,1)
81     -----RL3.forward(Conv3.output)
82     -----MP3.forward(RL3.output)
83
84     -----#flattening
85     -----F.forward(MP3.output)
86     -----x = F.output
87
88     -----
89     -----dense1.forward(x)
90     -----activation1.forward(dense1.output)
91     -----dense2.forward(activation1.output)
92     -----loss = loss_activation.forward(dense2.output, C)
93
94     -----predictions = np.argmax(loss_activation.output, axis = 1)
95     -----if len(C.shape) == 2:
96     -----C = np.argmax(C ,axis = 1)
97     -----accuracy = np.mean(predictions == C)
98
99     -----#backward pass
100     -----loss_activation.backward(loss_activation.output, C)
101     -----dense2.backward(loss_activation.dinputs)
102     -----activation1.backward(dense2.dinputs)
103     -----dense1.backward(activation1.dinputs)
104     -----F.backward(dense1.dinputs)
105     -----MP3.backward(F.dinputs)
106     -----RL3.backward(MP3.dinputs)@TBC
107
```
- Console (Bottom Panel)**: Shows the IPython prompt and the output of the executed script. The output is:

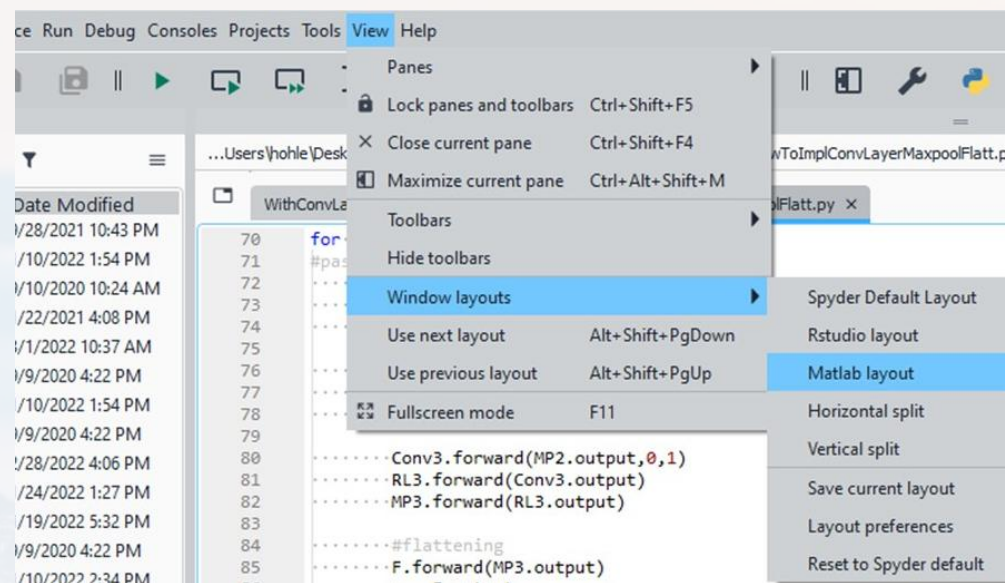
```
Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]:
```
- Workspace (Right Panel)**: Shows the current workspace with variables and plots. The workspace is currently empty.
- Help Panel (Bottom Right)**: Shows the usage instructions for the IDE.



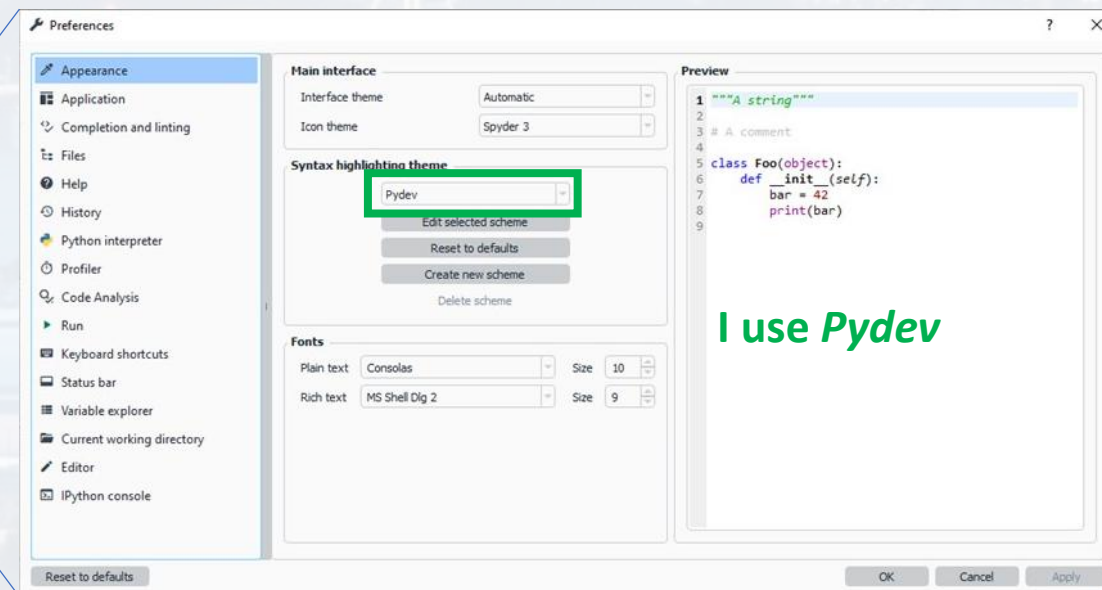
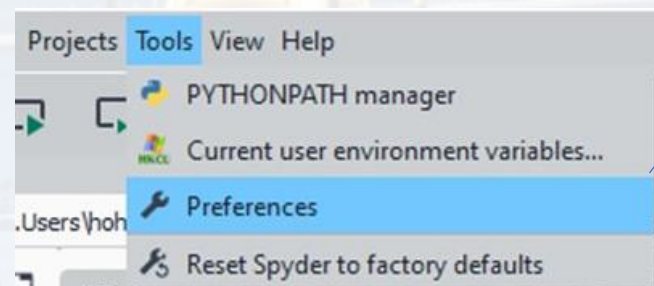
the environment
programming fundamentals
downloading UI
Spyder & Jupyter



settings:

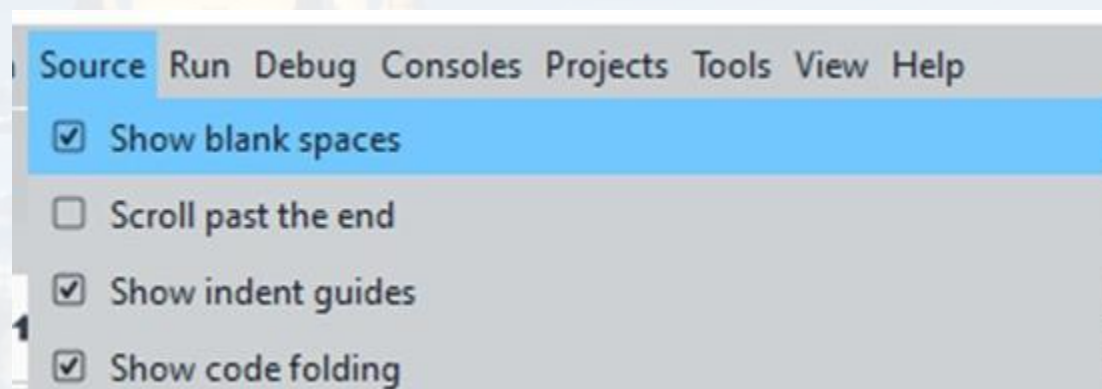
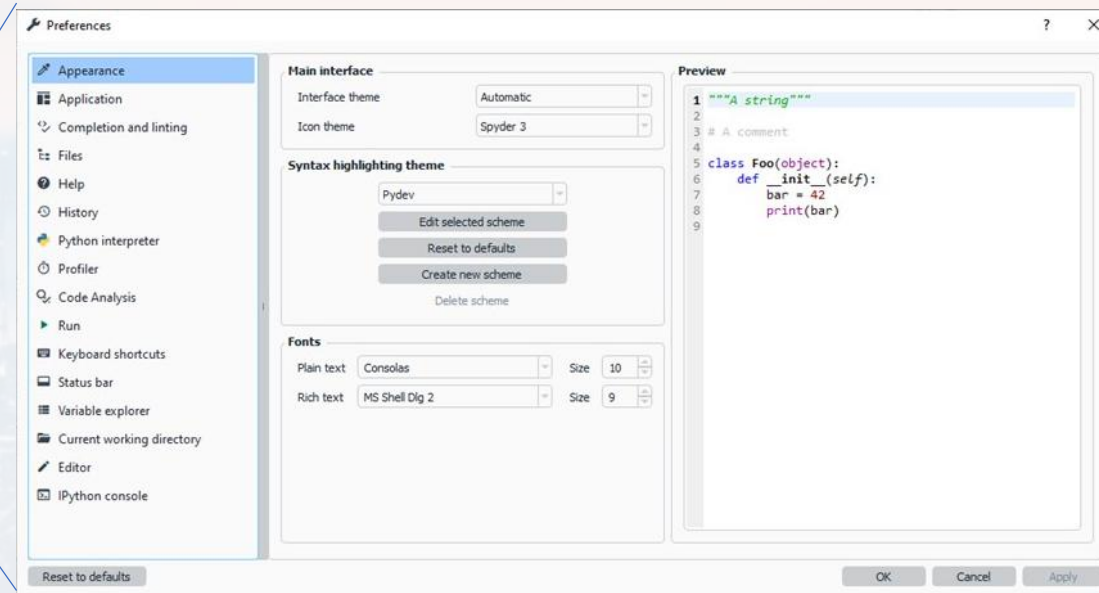
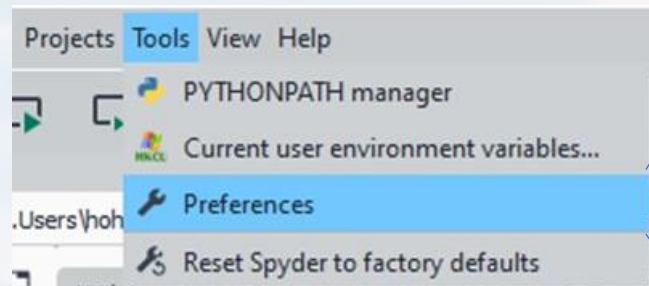
toolbar: *View* → *Window layouts*

e. g. Matlab

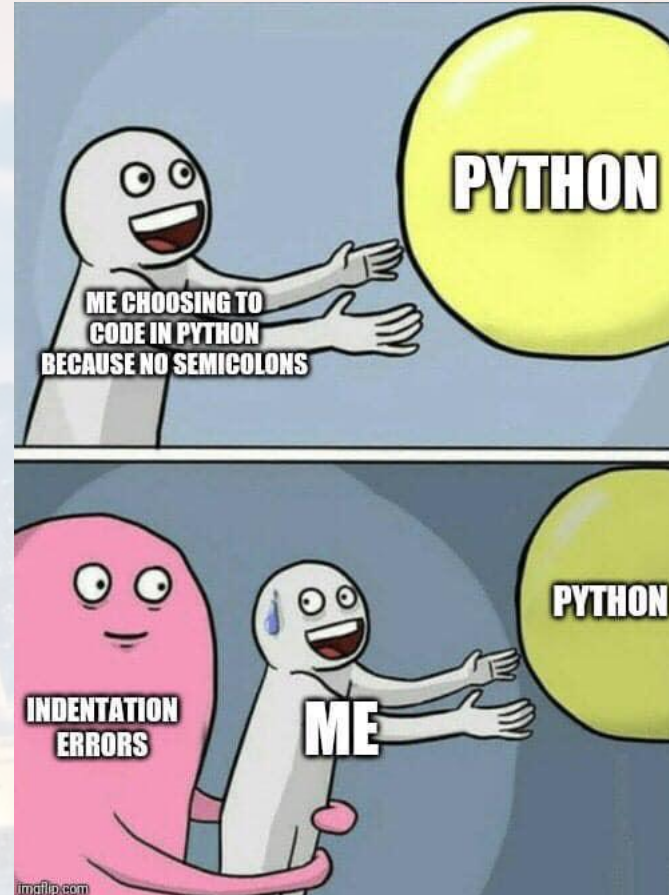




the environment
programming fundamentals
downloading UI
Spyder & Jupyter



spaces are relevant for syntax!



Thank you for your attention!